

Teste Técnico: Desenvolvedor Backend Python/Django - Pleno (Lojacorr)

1. Objetivo

O objetivo deste teste é avaliar sua proficiência na construção de APIs robustas utilizando Python e Django Rest Framework (DRF). Avaliaremos sua capacidade de modelagem, processamento de dados em lote, consumo de serviços externos, escrita de testes automatizados e padronização de ambiente com Docker.

2. O Desafio

Você deve desenvolver uma API RESTful para gerenciar um catálogo de Seguradoras, com enriquecimento de dados via API externa.

A. Modelagem e Endpoints:

- **POST** `/api/v1/seguradoras/importar/`: Recebe um JSON com uma lista de seguradoras (Nome, CNPJ, UF).
- **GET** `/api/v1/seguradoras/`: Listagem com paginação e filtros por UF e Nome.
- **Regra de Upsert**: Se o CNPJ já existir no banco, atualize os dados em vez de duplicar o registro.

B. Desafio de Integração (API Externa): O enriquecimento dos dados consultando a **BrasilAPI** (ou similar) **não deve bloquear** a resposta do endpoint de importação. O fluxo esperado é:

1. **Ação Imediata (POST)**: O endpoint `/api/v1/seguradoras/importar/` recebe a lista, salva os dados básicos fornecidos no payload de forma rápida no banco de dados e imediatamente devolve a resposta de sucesso (200/201) para o usuário.
2. **Ação em Background (Enriquecimento)**: A busca na API externa (para preencher os campos `nome_fantasia` e `situacao_cadastral` baseados no CNPJ) deve ocorrer fora do ciclo principal de request/response.
 - **Como implementar (sem Celery)**: Sinta-se livre para usar alternativas nativas e mais simples. Você pode, por exemplo, disparar uma *Thread* do Python no momento do salvamento, ou criar um *Django Management Command* (ex: `python manage.py enriquecer_seguradoras`) que varre os registros no banco e os atualiza. A escolha da estratégia arquitetural faz parte da avaliação.
 - **Endpoint sugerido**:
`https://brasilapi.com.br/api/cnpj/v1/{cnpj}`
 - **Tratamento de erro**: Caso a API externa esteja fora do ar ou o CNPJ não seja encontrado, a aplicação deve logar o erro e manter o registro apenas com os dados básicos, sem travar o sistema.

3. Requisitos Técnicos

- **Frameworks:** Python 3.11+ e Django 5.1 / DRF.
- **Banco de Dados:** PostgreSQL ou SQLite.
- **Docker (Obrigatório):** A aplicação deve possuir um `Dockerfile` e um `docker-compose.yml` que suba o servidor de aplicação e o banco (se for o caso) com um único comando.

4. Diferenciais (Pontos Desejáveis)

- **Dockerização Avançada:** Uso de *multi-stage builds* no Dockerfile para reduzir o tamanho da imagem.
- **Cache:** Implementar cache no endpoint de listagem.
- **Documentação:** Swagger/OpenAPI interativo (utilizando `drf-spectacular` ou `drf-yasg`).

5. Critérios de Avaliação

- **Arquitetura e Clean Code:** Organização do código, separação de responsabilidades e uso de boas práticas do DRF (Serializers, Viewsets).
- **Consumo de API:** Como você lidou com requisições HTTP externas (uso da biblioteca `requests` ou `httpx`), tratamento de timeouts e falhas.
- **Docker:** A facilidade de execução via Docker será avaliada. Falhas no build que demandem muito tempo de *troubleshooting* da nossa equipe impactarão negativamente a nota.
- **Integridade de Dados:** Validação correta de CNPJ e tratamento eficiente da regra de *Upsert* (evitando consultas excessivas ao banco).
- **Testes Automatizados:** Presença de testes para a lógica de importação e integração. **Espera-se o uso de Mocks** (ex: `unittest.mock` ou biblioteca `responses`) para simular a comunicação com a API externa e não depender de internet durante a suíte de testes.

6. Instruções de Entrega

1. Suba o código em um repositório público no GitHub ou GitLab.
2. O prazo sugerido para entrega é de **X dias corridos** [Definir o prazo, recomendo 7 dias] a partir do recebimento deste teste.
3. Garanta que seu `README.md` contenha:
 - Comando exato para subir o ambiente via Docker.
 - Instruções de como rodar os testes automatizados.
 - Exemplo de *payload* (JSON) para a rota de importação.
 - Breve explicação sobre a estratégia escolhida para desacoplar a chamada da API externa.

7. Aviso Importante (Disclaimer)

Sabemos que imprevistos acontecem e que o tempo disponível pode ser curto. **Caso você não consiga concluir 100% dos requisitos solicitados** (como finalizar os testes, a

dockerização ou a integração assíncrona), **envie o teste mesmo assim!** O código submetido será avaliado com base no que foi entregue, na qualidade da sua lógica e na organização da solução. Priorize entregar o que funciona bem e documente no README.md as dificuldades ou o que faltou concluir.

